

1-й Чекпоинт: ML System Design

1. Описание архитектуры сервиса

1.1 Общая архитектура

Сервис построен на основе **модульной архитектуры** с чёткой изоляцией компонентов. Каждый модуль отвечает за свою область ответственности, что обеспечивает гибкость, тестируемость и возможность независимой доработки.

1.2 Основные компоненты системы

Компонент	Ответственность	Технология
FastAPI Gateway	HTTP-сервер, валидация входа, маршрутизация запросов	FastAPI, Pydantic
Scanner	Рекурсивный поиск файлов, определение типов, дедупликация архивов	Python stdlib
Router	Маршрутизация по расширению файла	Python
Converters	Конвертация различных форматов в изображения или текст	cairosvg, lxml, zipfile, rarfile, py7zr, PyMuPDF
Preprocessing	Подготовка изображений: resize, контраст, инверсия фона	Pillow
OCR Module	Извлечение текстовых меток из изображений	Tesseract, pytesseract
Multimodal LLM	Понимание визуальной структуры + извлечение алгоритма	Qwen2-VL-2B, Transformers
Post-processing	Парсинг JSON, валидация, обработка ошибок	Pydantic, regex
Pipeline Orchestrator	Координация порядка шагов обработки	Python

1.3 Поток данных

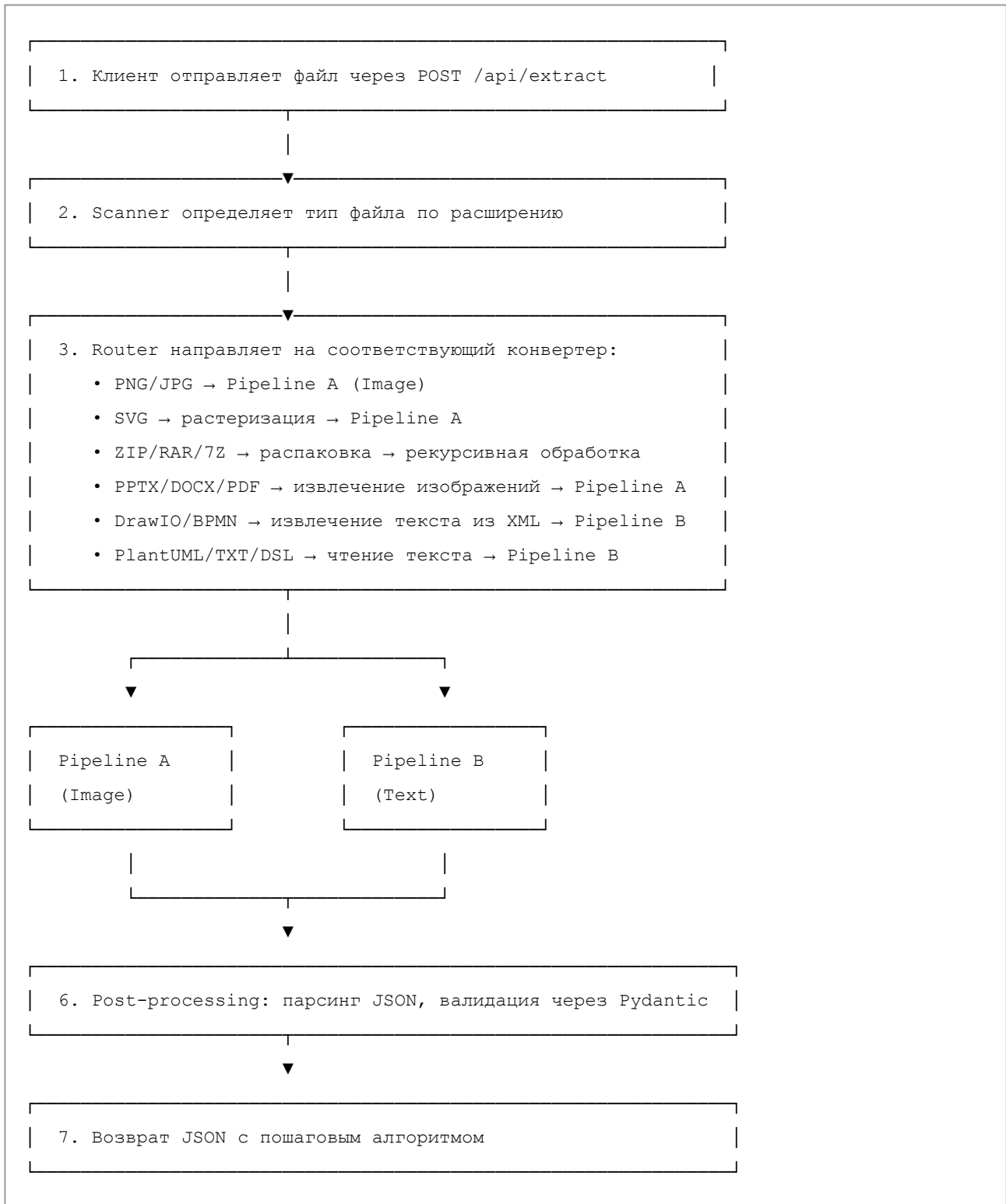
Вход

Файл изображения (PNG/JPG) или документ в одном из поддерживаемых форматов:

- **Изображения:** PNG, JPG, SVG
- **Текстовые форматы:** PlantUML (.puml, .plantuml), TXT, DSL
- **Структурированные:** DrawIO (.drawio), BPMN (.bpmn)

- **Документы:** PPTX, DOCX, PDF
- **Архивы:** ZIP, RAR, 7Z (распаковываются, содержимое обрабатывается рекурсивно)

Обработка



Pipeline A — Image Processing

```
файл → [конвертация если нужна] → preprocessing → OCR →  
→ Multimodal LLM (image + ocr_text) → postprocessing → JSON
```

Детали Pipeline A:

1. **Preprocessing:** resize, enhance контраст, инверсия тёмного фона
2. **OCR:** извлечение текста через Tesseract (rus+eng)
3. **Multimodal LLM:** анализ изображения + OCR-текст → JSON
4. **Для документов:** один файл → список изображений → каждое через Pipeline A отдельно

Pipeline B — Text Processing

```
файл → [извлечение текста из XML или read] →  
→ Text LLM prompt → postprocessing → JSON
```

Детали Pipeline B:

1. **DrawIO:** парсинг XML, извлечение атрибутов value из <mxCell>
2. **BPMN:** парсинг XML, извлечение participants, tasks, gateways, events
3. **PlantUML/TXT/DSL:** чтение содержимого как текст
4. **Text LLM:** анализ текстового описания → JSON

Выход

JSON с пошаговым алгоритмом:

```
{  
  "diagram_type": "BPMN",  
  "title": "Название процесса",  
  "steps": [  
    { "step": 1, "role": "Клиент", "action": "Оформляет заказ" },  
    { "step": 2, "role": "Система", "action": "Проверяет данные" },  
    { "step": 3, "role": "Система", "action": "Инициатирует оплату" }  
  ]  
}
```

1.4 Где используется ML/LLM/мультимодальная модель

1. Multimodal LLM (основной компонент)

Модель: Qwen2-VL-2B-Instruct

Функции:

- Понимание визуальной структуры диаграммы (стрелки, блоки, swim lanes)
- Извлечение семантики и последовательности шагов
- Генерация структурированного JSON-ответа
- Обработка изображений + OCR-текст как мультимодальный вход

Почему мультимодальная модель:

- Видит и понимает **визуальную структуру** (стрелки, направление потока)
- Понимает **текстовые метки** на диаграмме
- Генерализуется на **разные стили** диаграмм (BPMN, UML, hand-drawn)
- **Один компонент** вместо сложного CV-пайплайна (детекторы + OCR + graph construction)

2. OCR (вспомогательный компонент)

Технология: Tesseract

Функции:

- Извлечение текстовых меток из изображений
- Передача текста как дополнительный контекст для LLM
- Улучшение точности распознавания текста (особенно мелкий шрифт)

3. Text LLM (для текстовых форматов)

Технология: встроен в Qwen2-VL

Функции:

- Анализ PlantUML кода, DrawIO/BPMN XML, текстовых описаний
- Извлечение алгоритма из текстового представления

1.5 Как сервис будет разворачиваться

Контейнеризация

Технология: Docker + Docker Compose

Особенности:

- Base image: `python:3.11-slim`
- Все зависимости упакованы в образ
- Модель скачивается при первом запуске и кэшируется в volume
- Порт: 8000

Целевая среда

- **Основной сценарий:** CPU
- Требования к памяти: ~4-6 GB RAM

Запуск

```
docker compose up
```

Сервис поднимается и доступен через:

- `POST /api/extract` — обработка файла
- `GET /api/health` — проверка статуса

Масштабирование

- **Горизонтальное:** запуск нескольких контейнеров + load balancer
- **Вертикальное:** увеличение ресурсов контейнера

2. Технологический стек

Язык и фреймворк

Технология	Обоснование
Python 3.11	Развитая ML-экосистема; все ключевые библиотеки имеют Python-биндинги
FastAPI	Быстрый async HTTP-сервер; автогенерация OpenAPI; нативная поддержка async; легко тестируется
Pydantic v2	Валидация входных/выходных данных; автогенерация API-схемы; type safety

ML / CV / LLM

Технология	Обоснование
HuggingFace Transformers	Стандартная библиотека для загрузки и инференса open-source моделей
PyTorch	Нативная поддержка GPU и CPU; широкая экосистема
Qwen2-VL-2B-Instruct	2B параметров, Apache 2.0 лицензия, сильное многомодальное понимание, быстрее аналогов на CPU
Phi-3-vision-128k	Альтернатива: 4.2B параметров, MIT лицензия, лучшее качество (требуется GPU ≤8GB)
Tesseract + pytesseract	Локальная работа без внешних API, поддержка русского и английского языков

Обработка изображений и файлов

Технология	Обоснование
Pillow	Загрузка, resize, enhance, конвертация форматов
cairosvg	Конвертация SVG в растровые изображения
lxml	Быстрый парсинг DrawIO и BPMN XML-файлов

Технология	Обоснование
PyMuPDF (fitz)	Растеризация страниц PDF в изображения
zipfile (stdlib)	Извлечение изображений из PPTX/DOCX; обработка ZIP-архивов
rarfile	Распаковка RAR-архивов
py7zr	Распаковка 7Z-архивов

Инфраструктура

Технология	Обоснование
Docker + Docker Compose	Единый запуск, воспроизводимость окружения, изоляция зависимостей
REST API (JSON)	Стандартный формат для веб-сервисов, поддержка multipart/form-data

Поддерживаемые форматы входных данных

Формат	Количество	Метод обработки
PNG, JPG	~170 файлов	Pipeline A (Image)
SVG	5 файлов	Растеризация → Pipeline A
ZIP	10 архивов	Распаковка → рекурсивная обработка
RAR	1 архив	Распаковка → рекурсивная обработка
7Z	1 архив	Распаковка → рекурсивная обработка
DrawIO	18 файлов	XML парсинг → Pipeline B
BPMN	4 файла	XML парсинг → Pipeline B
PlantUML (.puml, .plantuml)	25 файлов	Pipeline B (Text)
TXT	15 файлов	Pipeline B (Text)
DSL	1 файл	Pipeline B (Text)
PPTX	2 файла (54 изображения)	Извлечение из ZIP → Pipeline A
DOCX	3 файла (9 изображений)	Извлечение из ZIP → Pipeline A
PDF	2 файла (~100 страниц)	Растеризация → Pipeline A

3. План реализации (Roadmap)

Этап 1 — Анализ задачи и данных

Задачи:

- Изучение всех трех наборов данных (155 PNG + 84 смешанных формата + документы)
- Определение целевого формата выхода: JSON со steps, roles, actions

- Baseline на 2-3 примерах вручную → зафиксирован "правильный ответ"
- Формулирование метрик оценки (Step Accuracy, Role Detection Rate, Latency)

Этап 2 — Выбор и первичная проверка модели

Задачи:

- Загрузка Qwen2-VL-2B-Instruct через HuggingFace Hub
- Минимальный inference-скрипт: загрузка изображения → промпт → генерация ответа
- Проверка на 2-3 примерах из датасета
- Замер времени инференса на CPU (целевой порог: ≤ 20 сек)
- Если скорость неудовлетворительна: 4-bit квантизация через bitsandbytes или GPTQ
- Если качество неудовлетворительно: переключение на Phi-3-vision (требует GPU)

Этап 3 — Реализация базового пайплайна

Задачи:

- Preprocessing: resize, enhance контраст
- OCR: извлечение текста через Tesseract; передача в модель как контекст
- Prompt engineering: few-shot промпт для structured JSON output
- Интеграция: image + OCR-text → модель → parse JSON → объект ответа
- Тестирование на примерах: BPMN, UML sequence, C4, hand-drawn
- Post-processing: валидация JSON; regex-fallback если JSON невалиден

Этап 4 — Оборачивание в API

Задачи:

- Создание FastAPI приложения
- Endpoint POST /api/extract — вход: multipart/form-data, выход: JSON
- Endpoint GET /api/health — проверка жизнеспособности сервиса
- Error handling: невалидный файл, превышение размера, timeout модели
- Logging: каждый запрос логируется (input hash, время, статус)
- Тестирование через curl и/или Postman

Этап 5 — Docker-сборка

Задачи:

- Dockerfile (multi-stage build для минимизации размера)
- docker-compose.yml: сервис + volume для кэша модели
- Тест полного цикла: docker compose up → curl запрос → ответ
- Оптимизация: .dockerignore, грамотная послойная стратегия кэша

Этап 6 — Валидация и оптимизация

Задачи:

- Прогон модели на всём датасете
- Сравнение с ground truth (test.txt + PlantUML спецификации)
- Расчёт метрик: Step Accuracy $\geq 70\%$, Role Detection Rate $\geq 60\%$
- Анализ ошибок: на каких диаграммах модель ошибается и почему
- Оптимизация промптов по результатам анализа
- Оптимизация скорости: квантизация, resize стратегия, lazy loading

Этап 7 — Обратная задача

Результат: Дополнительный endpoint: текст $\rightarrow$ диаграмма

Задачи:

- Endpoint `POST /api/generate` — вход: текстовое описание, выход: PNG
- LLM генерирует PlantUML-код из текстового описания
- PlantUML JAR рендерит код в изображение
- Тестирование: описание $\rightarrow$ код $\rightarrow$ рендер $\rightarrow$ визуальная проверка

Приоритеты реализации

КРИТИЧНЫЕ (этапы 1-5)	Без них нет работающего сервиса Основной фокус команды
ВАЖНЫЙ (этап 6)	Валидация качества Необходима для оценки результатов
ДОПОЛНИТЕЛЬНО (ЕСЛИ УСПЕЕМ) (этап 7)	Обратная задача

4. Метрики оценки качества

Метрика	Описание	Целевое значение
Step Accuracy	Доля шагов алгоритма, правильно извлечённых из диаграммы	$\geq 70\%$
Role Detection Rate	Доля шагов с правильно определённой ролью	$\geq 60\%$
Latency (P95)	Время обработки одного изображения (95-й перцентиль)	≤ 20 сек
Format Compliance	Доля ответов в валидном JSON-формате	100%

Метрика	Описание	Целевое значение
Multi-format Support	Количество поддерживаемых форматов входных файлов	13 форматов
Diagram Type Generalization	Работает ли на типах диаграмм, которых не было в примерах	Качественная оценка
